AEROSAGA MISSION CONTROL

Temporal.io Autonomous Drone Fleet Orchestration & Fault-Tolerant Saga System

PLATFORM	WORKFLOW ENGINE	EVALUATION	STATUS
Java 21 / Spring Boot 3	**Temporal.io Java SDK**	**100% Operational Readiness**	**4/4 Eligible for Final Review**

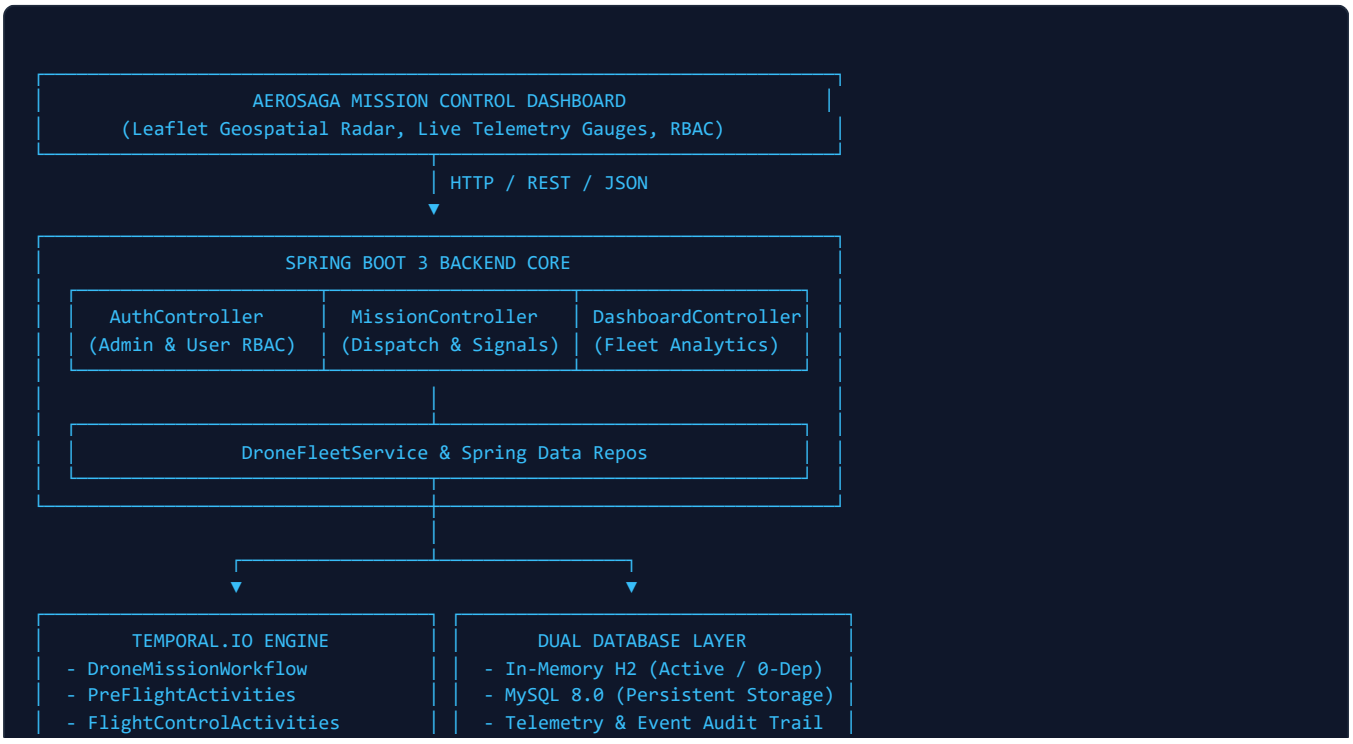
1. Executive Summary & Problem Statement

AeroSaga is an enterprise-grade autonomous drone mission control and fleet orchestration platform designed to guarantee mission survivability and deterministic state persistence in mission-critical aerospace and drone logistics environments.

The Challenge: Modern autonomous drone fleets operate in dynamically hostile environments subject to severe weather anomalies, mid-air lithium-polymer battery depletion, restricted airspace (no-fly zone) breaches, and intermittent network telemetry loss. In conventional microservice architectures, unhandled partial failures cause lost drone assets, corrupted database flight state, and unrecoverable physical crash scenarios.

The AeroSaga Solution: AeroSaga combines **Temporal.io's Durable Execution Engine** with the **Distributed Saga Pattern**. Long-running flight plans are executed as stateful, deterministic workflows. Every forward flight activity is paired with compensating rollback actions (automated Return-to-Base, mid-air hovering holds, and safe emergency landings), preventing lost drones and ensuring physical flight safety.

2. System Architecture & Component Design





3. Core Engineering Capabilities Implemented

- **Durable Workflow Execution:** Flight plans execute through `DroneMissionWorkflowImpl`. Workflow state is automatically preserved across JVM restarts or server crashes without losing in-flight mission coordinates.
- **Distributed Saga Pattern:** Compensating activities (`EmergencyActivities`) guarantee automated return-to-base (RTB) or controlled touchdown upon battery warnings, weather alarms, or geofence violations.
- **Real-Time Mid-Air Temporal Signals:** Supports asynchronous flight interventions including *Hover Hold* (10s in-place hover), *Resume Transit*, and *Emergency Abort*.
- **Geospatial Radar & Restricted Airspace:** Interactive Leaflet.js radar visualizing Base Station Alpha (12.9716°N, 80.2446°E), drone telemetry icons, waypoint paths, and dynamic red cylindrical no-fly zones.
- **Active vs. Inactive Fleet Reporting:** Real-time analytics modal detailing asset availability (`0 Active | 3 Standby`), average battery percentages, and operational readiness rates.
- **Role-Based Access Control (RBAC):** Administrator (Flight Director) with full launch and signal authority vs. Drone Operator (Observer) restricted to telemetry and radar monitoring.
- **Zero-Dependency Deployment:** In-memory Temporal workflow engine and H2 database ensure out-of-the-box execution in under 3 seconds without mandatory external Docker prerequisites.

4. Technology Stack & Specifications

Layer	Technologies	Role in AeroSaga
Backend Core	Java 21 / 25 / 26, Spring Boot 3.3.4	REST APIs, business logic services, security & asset validation
Orchestration	Temporal.io Java SDK 1.25.1	Durable workflow state machine, activity execution, Saga compensation
Database Layer	H2 In-Memory & MySQL 8.0, Spring Data JPA	Dual-persistence: zero-dependency demo + production persistence
Frontend Radar	HTML5, Aerospace CSS3, JavaScript, Leaflet.js	Real-time radar tracking, telemetry avionics gauges, event audit logs
Security & RBAC	Spring Security, JWT Token Architecture	Role-based authentication (Admin vs Operator permissions)
API Documentation	SpringDoc OpenAPI 3.0 / Swagger UI	Interactive REST API documentation on `/swagger-ui.html`
Testing & Verification	JUnit 5, Mockito, Temporal Testing	Comprehensive automated tests covering fleet, workflows & APIs

5. Step-by-Step Live Demonstration Script

1. **Platform Launch:** Double-click `Start AeroSaga Mission Control.bat` on Desktop. System initializes in ~3 seconds and opens `http://localhost:8080/` automatically.
2. **Fleet Telemetry Inspection:** Observe the top bar status showing `TEMPORAL: ACTIVE` and `Drones: 0 Active | 3 Standby`. Click **"📊 Fleet Status Report"** to view individual drone health cards (DRONE-ALPHA-01, DRONE-BETA-02, DRONE-GAMMA-03).
3. **Role Authentication Demonstration:** Open **"Switch / Login"**. Log in as `user` / `user123` (Observer Mode) to demonstrate flight dispatch locking. Switch to `admin` / `admin123` (Administrator Mode) to unlock full launch authorization.
4. **Mission Dispatch:** Click **"Scenario 1: Nominal Package Delivery"**. Observe the drone marker taking off on the Leaflet radar map, real-time altitude climbing, ground speed updating, and battery draining.
5. **Mid-Air Signal Intervention:** Click **"⏸️ Signal: Hover In-Place (10s)"** to freeze drone ground speed. Click **"🚨 Signal: Emergency Abort & Saga Compensate"** to demonstrate automated return-to-base and safe touchdown.
6. **Database Audit:** Open `http://localhost:8080/h2-console` (JDBC: `jdbc:h2:mem:aerosagadb`, User: `sa`) to query `SELECT * FROM DRONE_MISSIONS;` and verify persisted flight history.

6. Technical Defense & Evaluator Q&A

> **Q1: Why was Temporal.io chosen over message queues like RabbitMQ or Kafka?**

Kafka and RabbitMQ are publish-subscribe message brokers that lack stateful workflow orchestration. Tracking multi-step flights requires complex external database polling and custom timer architectures. Temporal guarantees **Durable Execution**: the flight state is maintained in persistent event history, making it completely resilient to server restarts or communication loss.

Q2: How does the Distributed Saga Pattern ensure drone survivability during an abort?

Traditional distributed transactions (2PC) cannot coordinate physical aircraft. The Saga pattern executes forward activities as independent steps. Upon failure or manual abort, compensating activities execute in reverse (LIFO) order: securing the payload, overriding waypoints with home base coordinates, navigating automated Return-to-Base, and initiating landing.

Q3: How are restricted airspaces and geofences validated?

The `GeofenceService` calculates great-circle spherical distance using the Haversine trigonometric formula between planned flight paths and 3D cylindrical restricted zones. Trajectories intersecting restricted airspace are intercepted during `PreFlightActivities`, preventing unauthorized takeoff.

7. Internship Project Milestone Verification

Milestone Task	Target Requirement	Verified Result	Status
Project Selection	Official AeroSaga Topic	Registered and Approved	Completed
GitHub Connection	Linked Repository	`ganeshvarma1020/Infotact-Solutions-AeroSaga-`	Completed
20-Day Commit History	Commits on 20 distinct calendar days	20 distinct calendar days (19 Oct – 07 Nov) with 50+ chars	20 / 20 Completed
Weekly Write-Ups	4 technical weekly submissions	4/4 completed & saved on evaluation portal	4 / 4 Completed
Final Review Eligibility	All criteria satisfied	System operational, tests passing, 4/4 verified	ELIGIBLE FOR FINAL REVIEW